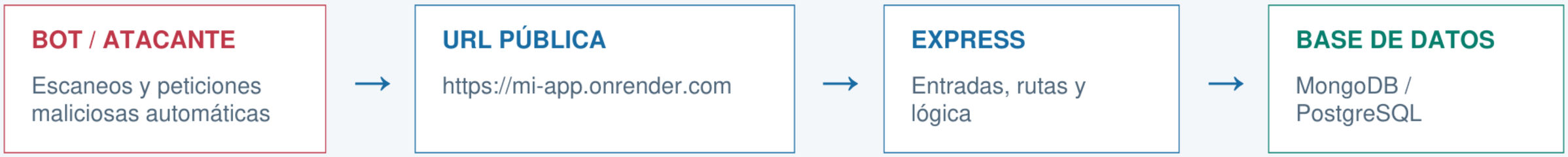
Anatomía de las amenazas web

Qué cambia cuando Node.js + MongoDB / PostgreSQL sale de localhost a Internet.



01 Inyecciones

Entradas que alteran consultas y exponen o destruyen datos.

02 Secretos expuestos

URI, contraseñas o tokens en Git, logs o imágenes.

03 Denegación de servicio

Exceso de peticiones o trabajo costoso: CPU, memoria, conexiones.

04 Dependencias vulnerables

Fallos conocidos y paquetes comprometidos en la cadena npm.

05 Tráfico sin TLS

Credenciales y datos legibles para quien intercepte la conexión.

Exposición pública = posibilidad de escaneo inmediato.
Cinco riesgos relevantes, no cinco categorías literales del OWASP Top 10.

La seguridad se diseña desde la primera línea de código.

Inyecciones: NoSQL frente a SQL

La entrada del usuario debe conservar su papel de dato, sin convertirse en una instrucción.

MongoDB / Mongoose

```
{ "newItem": { "$gt": "" } }
```

Si se incorpora como selector, el objeto puede ampliar la búsqueda. El efecto depende de la consulta.

```
const { newItem } = req.body;
if (typeof newItem !== 'string' ||
    newItem.length > 200) {
  return res.sendStatus(400);
}
await Item.find({ name: newItem })
  .setOptions({ sanitizeFilter: true });
```

Esquemas estrictos + campos permitidos. La sanitización es una defensa adicional con versiones actualizadas.

PostgreSQL / node-postgres

```
// VULNERABLE: id = "1 OR 1=1"
pool.query(
  "DELETE FROM items WHERE id = "
  + req.body.id
);
```

```
// VALORES SEPARADOS DEL SQL
await pool.query(
  "DELETE FROM items WHERE id = $1",
  [req.body.id]
);
```

Validar también el id y autorizar la operación.
$1 no parametriza nombres de tabla o columna.

Validar tipos y campos en MongoDB. Parametrizar valores en PostgreSQL.

Gestión de secretos y mínimo privilegio

Credenciales diferentes para desarrollo, pruebas y producción.

NIVEL 0

CÓDIGO / GIT

Contraseña en awcodeAtlas.js
o awcodePostgres.js.
Exposición crítica.

NIVEL 1

LOCAL

.env fuera de Git.
.env.example sin secretos.
Acceso local restringido.

NIVEL 2

CLOUD

Variables desde Render,
Railway o Fly.io.
Sin copiarlas a logs.

NIVEL 3

BÓVEDA

Acceso por identidad,
auditoría y rotación.
Sin claves en la imagen.

Bóvedas e integración

AWS Secrets Manager / Parameter Store
Google Secret Manager / Azure Key Vault
HashiCorp Vault · Sealed Secrets / CSI

Una identidad limitada para la app

MongoDB: readWrite solo sobre todolistDB.
PostgreSQL: rol sin superusuario, con permisos
solo sobre el esquema y las tablas necesarias.
Migraciones: identidad separada cuando sea posible.

Si un secreto se publica, se revoca o rota. Borrarlo de Git no basta.

Hardening de Express

Cabeceras, límites de peticiones y acceso desde el navegador.

```
import helmet from 'helmet';
import cors from 'cors';
import { rateLimit } from 'express-rate-limit';

app.disable('x-powered-by');
app.use(helmet({
  xFrameOptions: { action: 'deny' }
}));
app.use(cors({
  origin: 'https://mi-app.com'
}));
app.use(rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 100,
  message: 'Demasiadas peticiones'
}));
```

Helmet

CSP limita recursos y scripts.
HSTS indica al navegador que use HTTPS.
nosniff evita inferencias de tipo MIME.
DENY impide iframes en este ejemplo.

Límite de ejemplo: 100 / 15 min

Hasta el límite: la petición continúa.
Al superarlo: respuesta HTTP 429.
Ajustar por endpoint y tráfico real.

En producción

Configurar trust proxy según la topología.
Compartir contadores entre réplicas.
Añadir protección perimetral frente a DDoS.

CORS controla la lectura en navegadores. La API sigue necesitando autenticación y autorización.

Contenedores: mínimo privilegio y escaneo

Una imagen pequeña, mantenida y reproducible reduce la superficie de ataque.

Patrones que abren la puerta

node:latest cambia sin control.
root amplía el impacto de un fallo.
COPY sin filtro puede incluir secretos.
Dependencias de desarrollo innecesarias.

.dockerignore

```
.env
.env.*
!.env.example
.git
node_modules
README.md
*.log
*.tmp
```

Dockerfile de referencia

```
FROM node:24-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY --chown=node:node . .
USER node
CMD ["node", "awcodeAtlas.js"]
```

Fijar versión / digest y reconstruir con parches.
Para PostgreSQL: cambiar el archivo de arranque.

```
trivy image --exit-code 1 \
  --severity CRITICAL \
  mi-usuario/todo-app:1.0.0
```

La aplicación se ejecuta sin root. Los secretos se inyectan al arrancar.

Seguridad de red: TLS, WAF y base de datos

Cifrado en cada salto y exposición limitada a los orígenes necesarios.



TLS en cada enlace

Preferir TLS 1.3 cuando se soporte.
Validar certificado y nombre del servidor.
Certificados gestionados: Let’s Encrypt / ACM.

WAF en la frontera

Cloudflare / AWS WAF / Cloud Armor.
Reglas contra patrones e IPs sospechosas.
Ajustar falsos positivos y evitar bypass del origen.

Acceso público restringido

Atlas: permitir las IPs fijas de salida del backend.
Evitar 0.0.0.0/0 permanente.
PostgreSQL: firewall y reglas de acceso ajustadas.

Acceso privado

VPC Peering o Private Endpoints / PrivateLink.
Comprobar disponibilidad según proveedor y plan.
Separar credenciales y redes de cada entorno.

Un certificado en el balanceador no cifra automáticamente los siguientes saltos.

Kubernetes y OpenShift: contención del daño

Security Context, Network Policies y permisos mínimos para cada carga.

Dentro de containers[]

```
securityContext:
  runAsNonRoot: true
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]
  seccompProfile:
    type: RuntimeDefault
```

OpenShift puede asignar un UID arbitrario.
Evitar fijar 1000 sin comprobar las SCC.
Montar un volumen limitado si hace falta /tmp.

RBAC y admisión

Sin cluster-admin. Pod Security restricted en K8s.
SCC en OpenShift y ServiceAccount con permisos mínimos.

INGRESS CONTROLLER

Pods / namespaces autorizados

↓ **Solo puerto de la app**

POD DE LA APP

Ingreso y egreso con default-deny

↓ **Solo destinos explícitos**

MONGO 27017 / PG 5432

Además: DNS TCP/UDP 53 cuando sea necesario

Desarrollo × Base de datos de producción

El CNI debe aplicar NetworkPolicy. Se seleccionan pods, namespaces o IPs, no Services. Atlas dinámico puede requerir egreso por FQDN específico del CNI.

Denegar por defecto y permitir solo los flujos necesarios limita el movimiento lateral.

DevSecOps: puertas de seguridad en CI/CD

Cada cambio pasa controles antes de promover la imagen a producción.

1 **SECRETOS**

Gitleaks / TruffleHog

Antes del push y en CI.
Un hook local puede omitirse.

→ **2** **DEPENDENCIAS**

npm audit / Snyk

SCA del árbol resuelto.
Actualizar y revisar hallazgos.

→ **3** **CÓDIGO**

SAST + reglas ESLint

Patrones de riesgo.
Añadir revisión y pruebas.

4 **IMAGEN**

Trivy / Docker Scout

Paquetes del contenedor.
Umbral que hace fallar CI.

→ **5** **FIRMA**

Cosign / Sigstore

Firmar el digest. Verificar
identidad al desplegar.

→ **6** **DESPLIEGUE**

Gates obligatorios

Solo artefactos aprobados.
Fallo crítico: parar y avisar.

FALLO CRÍTICO **El job devuelve error y bloquea la promoción. Las excepciones tienen responsable y caducidad.**

Una puerta solo protege si su fallo detiene el pipeline y alguien atiende la alerta.

Checklist y matriz de madurez

Una comprobación repetible antes de cada despliegue.

- ☐ Secretos fuera de Git, logs e imágenes
- ☐ SQL parametrizado / NoSQL con validación estricta
- ☐ Helmet activo y política CSP revisada
- ☐ Rate limiting ajustado y probado
- ☐ Contenedor sin root y dependencias necesarias
- ☐ TLS con certificados verificados en cada salto
- ☐ Base de datos accesible solo desde orígenes autorizados
- ☐ Auditoría periódica y hallazgos críticos resueltos

Niveles acumulativos

Nivel	Controles añadidos
1 · Básico	.env fuera de Git HTTPS + TLS a la BD Consultas seguras
2 · Profesional	Helmet + rate limiting Contenedor sin root CI con escaneo
3 · Corporativo	WAF + bóveda Políticas K8s / OpenShift Firma y verificación en CI/CD

También imprescindibles

Autenticación y autorización, copias restaurables, monitorización y respuesta a incidentes.

La seguridad requiere mantenimiento continuo, evidencias y pruebas de recuperación.